TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesor:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: María José Balcázar Loor

PARALELO: A

SEMESTRE: 5to

ACTIVIDAD: Estudio de caso – Sistema de control de rescate y adopción animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad ____

Actividad # ____1____

ESTUDIO DE CASO

Sistema de Control de Rescate y Adopción Animal

1. Introducción

El rescate y la adopción de animales requieren una gestión organizada de la información para facilitar el seguimiento de cada caso y la coordinación entre las personas responsables. El registro de los animales, la asignación de voluntarios y veterinarios y el control de los estados del proceso son actividades importantes para mantener la información actualizada y mejorar la organización del refugio.

En este contexto, el presente trabajo propone el diseño de la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal, destinado a gestionar el ingreso de perros, gatos y fauna silvestre, así como la asignación del personal encargado y el seguimiento de las etapas de atención.

Para estructurar la solución se plantean los requerimientos funcionales y no funcionales, los actores que interactúan con el sistema y los diagramas del Lenguaje Unificado de Modelado (UML). Además, se propone aplicar los patrones de diseño Singleton, Observer, Factory Method y Modelo-Vista-Controlador (MVC), con el propósito de organizar las responsabilidades del software y facilitar su mantenimiento. La utilización de UML permite representar de manera visual la estructura y el comportamiento de la solución propuesta (Object Management Group, s. f.).

Finalmente, se contempla el uso de GitHub y el modelo Gitflow para organizar las ramas de desarrollo, registrar los cambios y mantener un historial verificable del proyecto.

2. Objetivos

2.1. Objetivo general

Diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal mediante el análisis de requerimientos, la elaboración de diagramas UML, la aplicación de patrones de diseño de software y la utilización del modelo Gitflow, con el fin de proponer una solución organizada, reutilizable y mantenible.

2.2. Objetivos específicos

- Identificar los actores principales del sistema y establecer los requerimientos funcionales y no funcionales necesarios para la gestión de los casos de rescate y adopción.

- Representar la estructura y el comportamiento del sistema mediante diagramas UML, considerando las relaciones entre sus componentes y los principios SOLID.
- Seleccionar y justificar patrones de diseño que permitan organizar la creación de expedientes, las notificaciones y la separación de responsabilidades.
- Proponer una estructura de ramas Gitflow en un repositorio público de GitHub para organizar los avances y conservar la trazabilidad de las modificaciones.

3. Planteamiento del problema

La gestión de un refugio animal requiere registrar los ingresos de animales rescatados, identificar su especie, documentar su situación y asignar al personal responsable de su atención. Asimismo, es necesario actualizar el estado de cada caso conforme avanza el proceso, desde el rescate inicial hasta el tratamiento veterinario y la posible adopción.

Cuando estas actividades no cuentan con una organización adecuada de la información, pueden presentarse dificultades para consultar los expedientes, conocer las responsabilidades asignadas y dar seguimiento a las actividades pendientes. Además, la falta de un procedimiento definido para notificar nuevos casos puede dificultar la coordinación del personal.

Ante esta situación, se propone diseñar un sistema que centralice la información de los animales rescatados y permita gestionar sus expedientes, asignaciones y estados. La arquitectura planteada también contempla mecanismos de notificación y una estructura de desarrollo basada en patrones de diseño y control de versiones.

4. Alcance del sistema

El sistema propuesto estará orientado a la gestión interna de los casos de rescate y adopción animal de un refugio. Sus funcionalidades principales comprenderán:

- Registrar nuevos casos de rescate con información básica del animal, fecha, ubicación y descripción de la situación.
- Clasificar los expedientes según la especie: perro, gato o fauna silvestre.
- Asignar voluntarios o veterinarios responsables de cada caso.
- Consultar y actualizar el estado del proceso, incluyendo Rescatado, En Tratamiento y Adopción.
- Generar notificaciones para informar al personal correspondiente sobre nuevos casos.
- Mantener organizada la información de los expedientes y sus cambios.

El diseño contempla una arquitectura lógica y una propuesta de organización del código. No se considera que exista una plataforma completamente operativa hasta que las funcionalidades correspondientes hayan sido desarrolladas y verificadas.

5. Análisis de requerimientos

5.1. Requerimientos funcionales

Los requerimientos funcionales describen las acciones que el sistema debe permitir realizar.

RF-01. Registro de casos: el sistema deberá permitir registrar un nuevo caso de rescate con un identificador, fecha de ingreso, ubicación y descripción.

RF-02. Clasificación de animales: el sistema deberá permitir seleccionar la especie del animal entre perro, gato y fauna silvestre.

RF-03. Registro de información del animal: el sistema deberá permitir almacenar datos como nombre provisional, características físicas y observaciones relevantes, cuando estén disponibles.

RF-04. Asignación de responsables: el sistema deberá permitir asignar un voluntario o veterinario a un caso registrado.

RF-05. Gestión de estados: el sistema deberá permitir actualizar y consultar el estado de cada expediente. Entre los estados iniciales se contemplan Rescatado, En Tratamiento y Adopción.

RF-06. Notificaciones: el sistema deberá contemplar un mecanismo para notificar a los responsables correspondientes cuando se registre un nuevo caso.

RF-07. Consulta de expedientes: el sistema deberá permitir consultar la información y el estado actual de los casos registrados.

RF-08. Seguimiento de cambios: el diseño deberá contemplar el registro de las actualizaciones relevantes de los expedientes para facilitar su seguimiento.

5.2. Requerimientos no funcionales

Los requerimientos no funcionales establecen las características de calidad que debe considerar la solución.

RNF-01. Mantenibilidad: el sistema deberá organizar sus componentes de manera que las modificaciones puedan realizarse sin afectar innecesariamente otras funcionalidades.

RNF-02. Escalabilidad: la arquitectura deberá permitir incorporar nuevas especies, funcionalidades o tipos de notificación con cambios controlados.

RNF-03. Trazabilidad: el desarrollo deberá utilizar Git y GitHub para conservar un historial de cambios identificable mediante commits y ramas.

RNF-04. Reutilización: los componentes deberán diseñarse para aprovechar comportamientos comunes y evitar duplicaciones innecesarias.

RNF-05. Organización y legibilidad: las clases, métodos y módulos deberán utilizar nombres significativos y responsabilidades claramente definidas.

RNF-06. Integridad de la información: el sistema deberá validar los datos obligatorios antes de registrar un caso y evitar asignaciones o actualizaciones inconsistentes.

6. Actores principales

Ciudadano o reportante: representa a la persona que comunica una situación de rescate y proporciona los datos disponibles del animal. En el alcance propuesto, su función principal es aportar información para el registro del caso.

Voluntario: representa al integrante del refugio encargado de atender los casos que le sean asignados y colaborar en el seguimiento del animal.

Veterinario: representa al profesional responsable de evaluar la condición de salud del animal, registrar observaciones y documentar el tratamiento correspondiente.

Administrador del refugio: representa al usuario responsable de supervisar los expedientes, organizar las asignaciones y mantener el control de la información.

La identificación de estos actores servirá como base para elaborar el diagrama de casos de uso y definir las interacciones entre los usuarios y la plataforma.

Diagramas

Diagrama de casos de uso

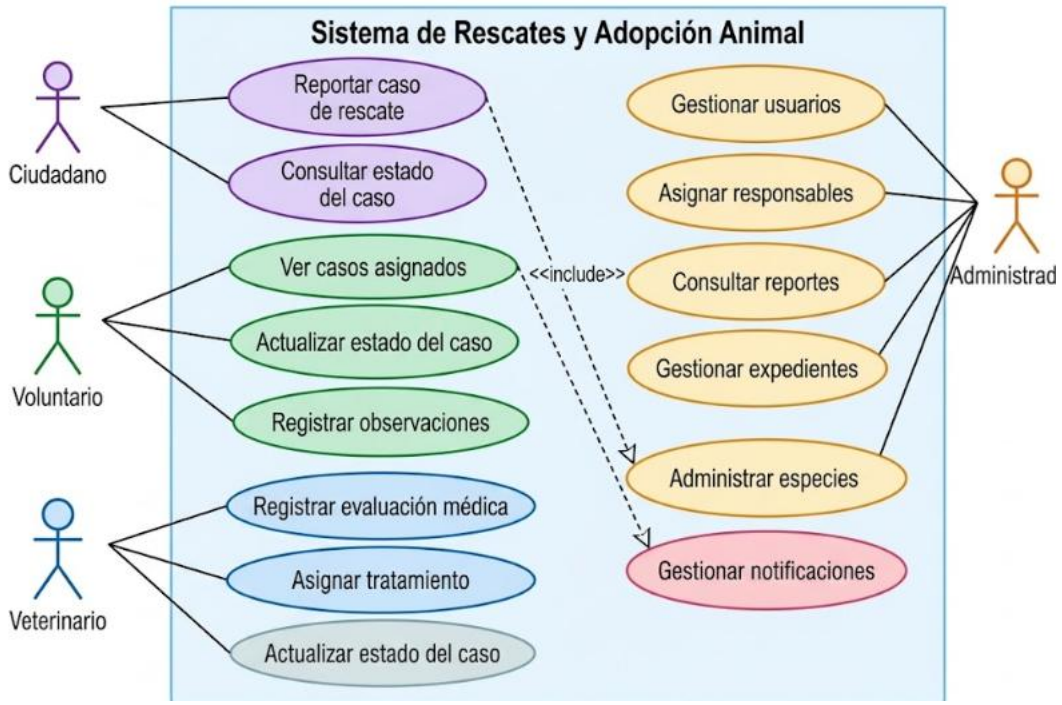


Diagrama de clases UML

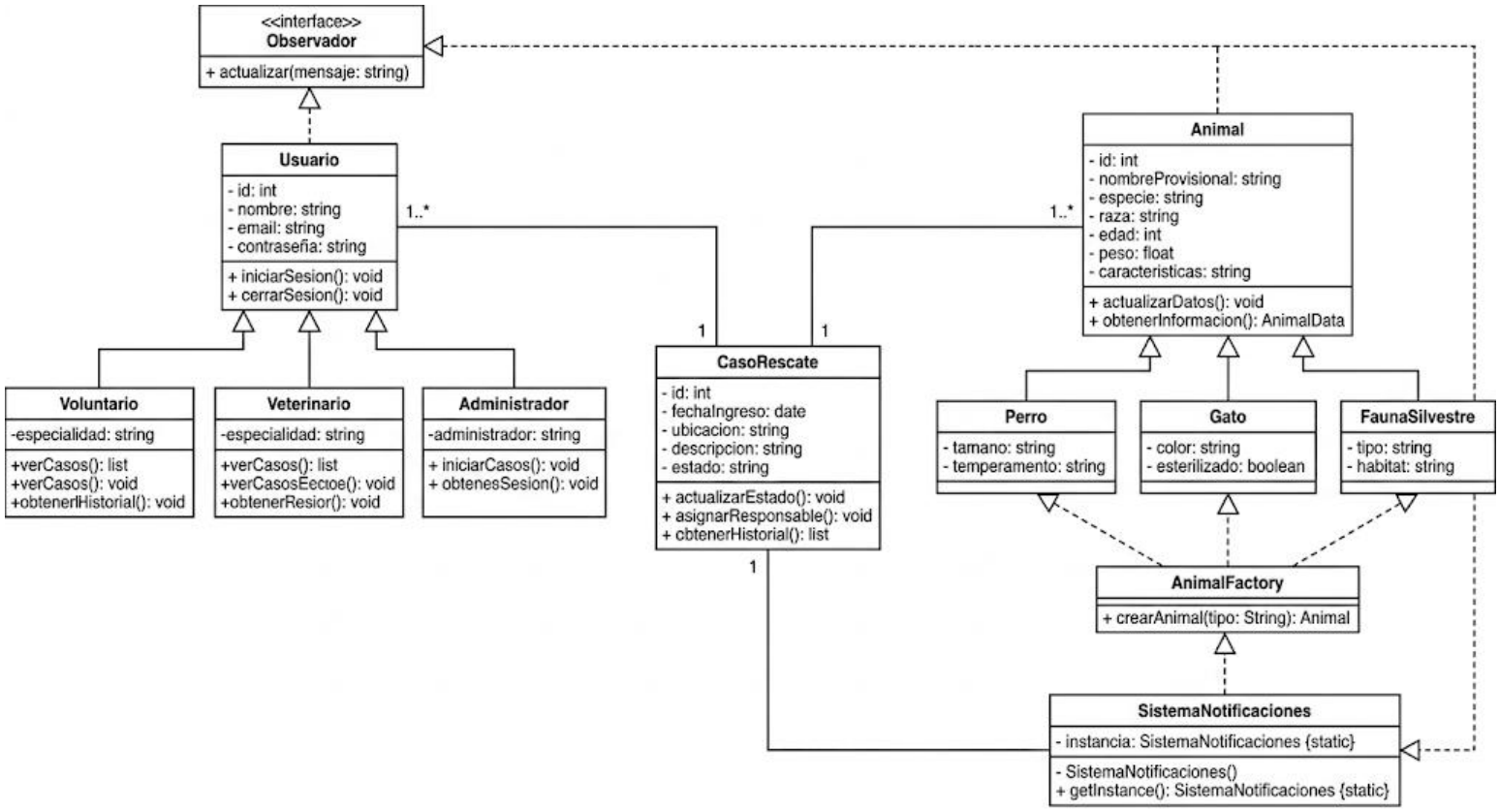
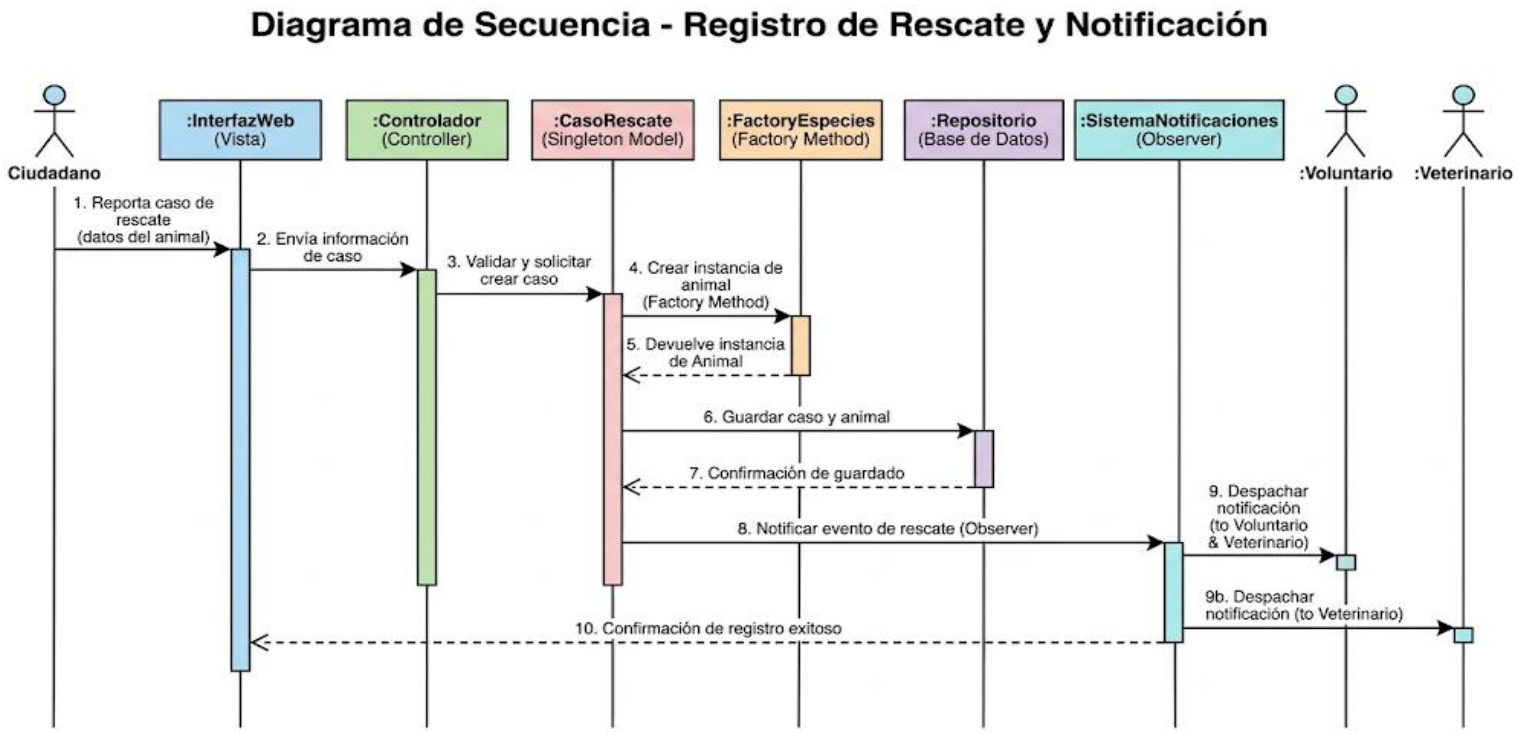
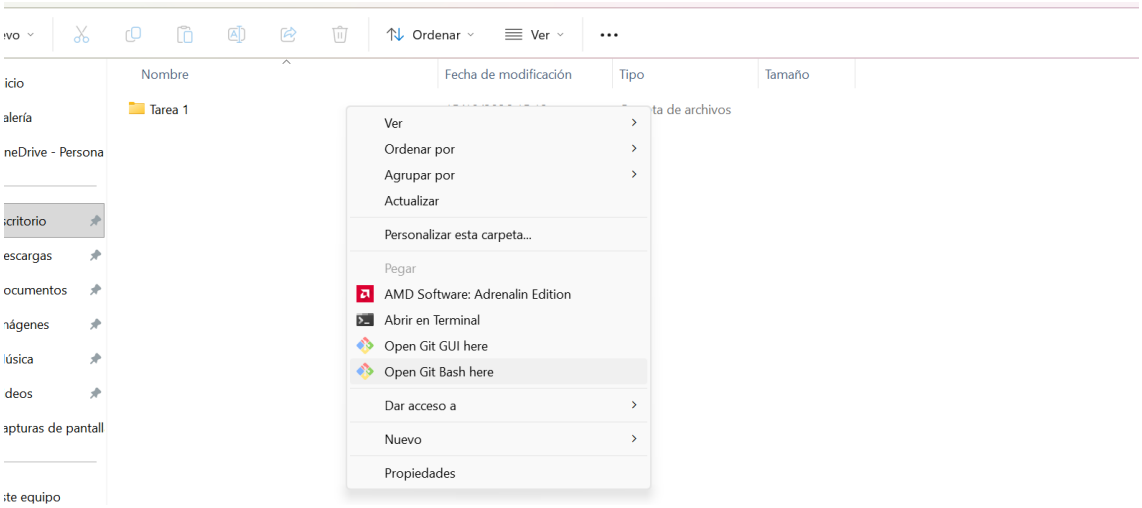
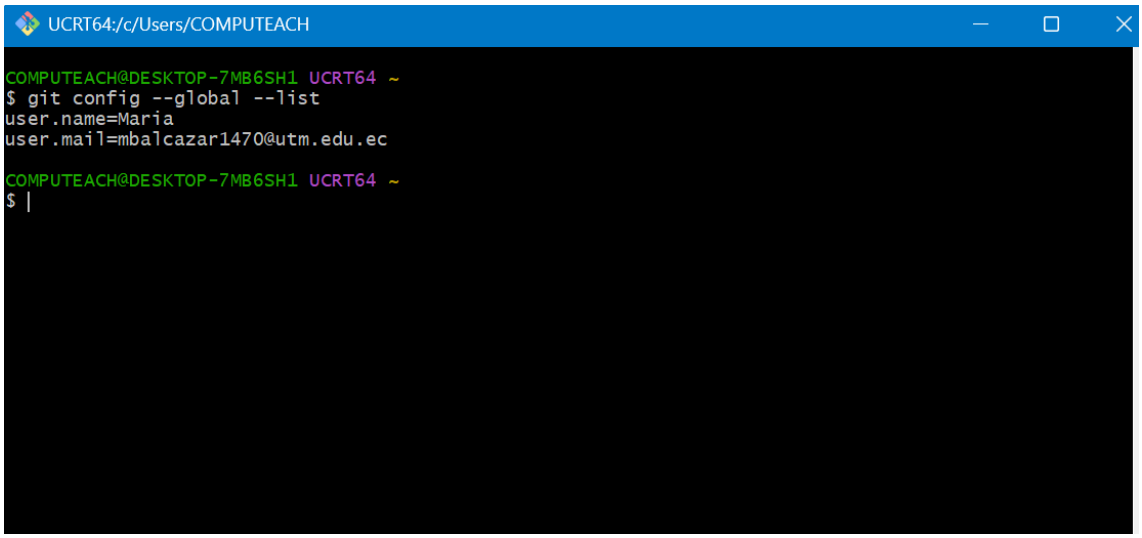


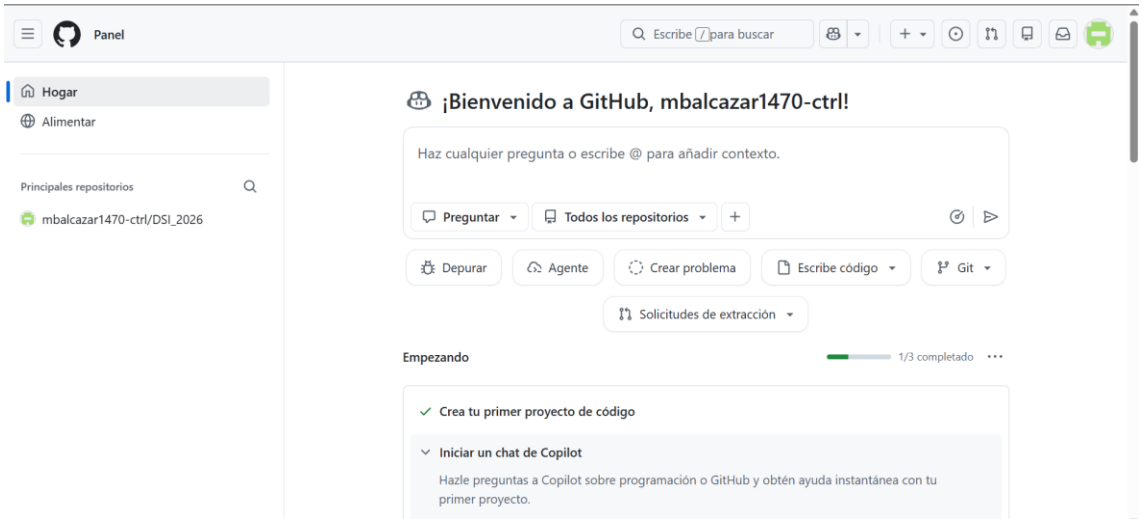
Diagrama de Secuencia

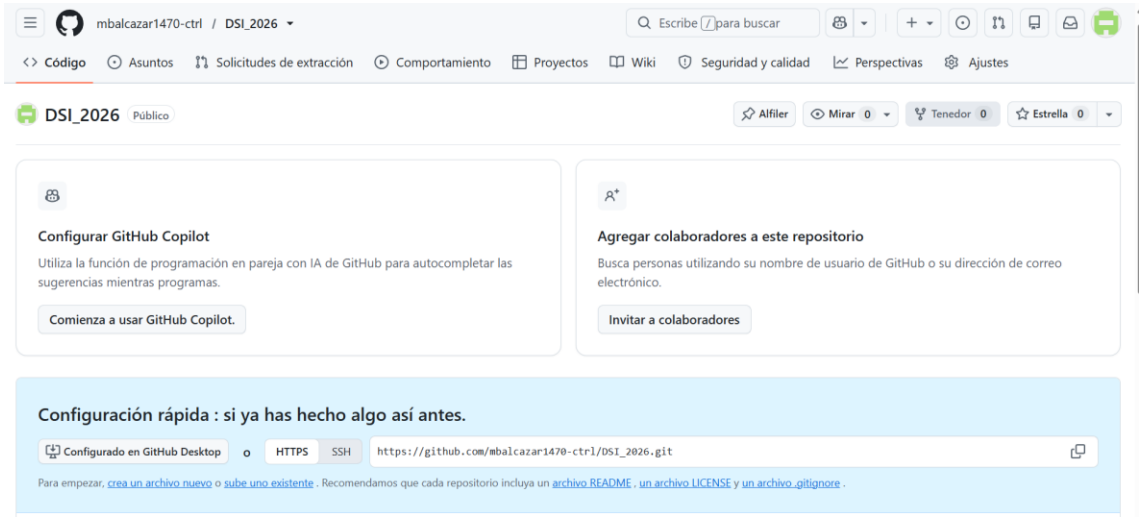


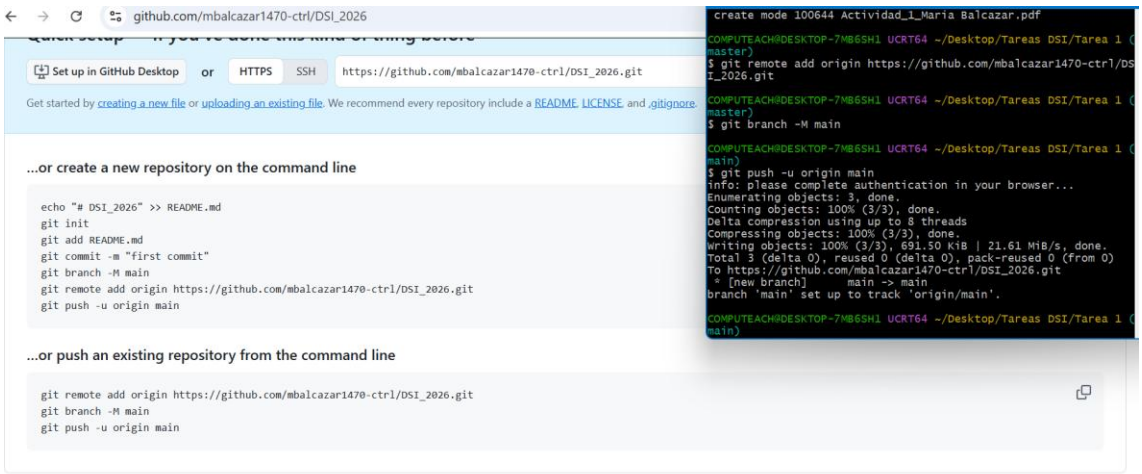
Evidencias

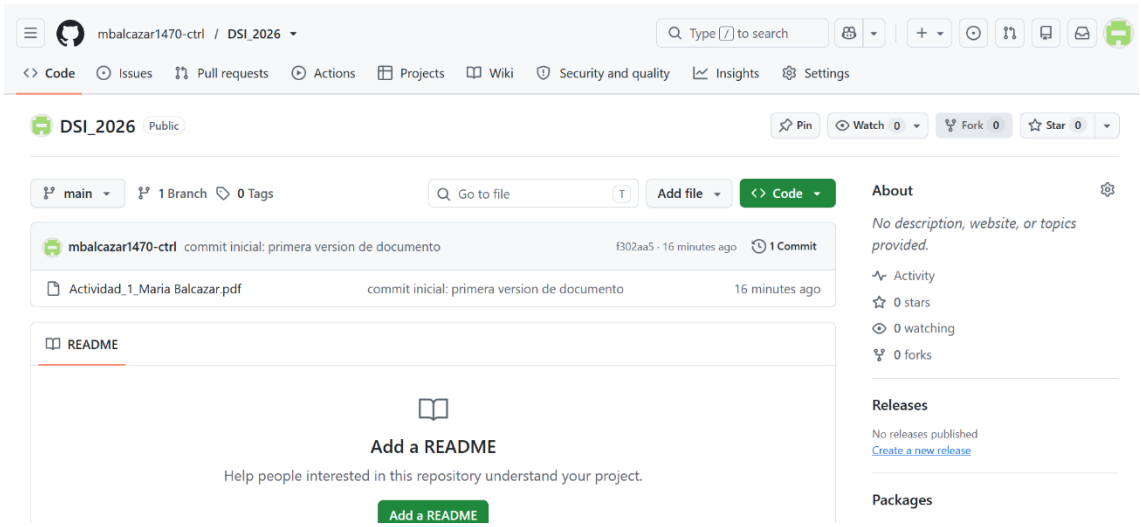

Ordenar
Ver
Nombre
Fecha de modificación
Tipo
Tamaño
Tarea 1
Ver
Ordenar por
Agrupar por
Actualizar
Personalizar esta carpeta...
Pegar
AMD Software: Adrenalin Edition
Abrir en Terminal
Open Git GUI here
Open Git Bash here
Dar acceso a
Nuevo
Propiedades


UCRT64:/c/Users/COMPUTEACH
COMPUTEACH@DESKTOP-7MB6SH1 UCRT64 ~
$ git config --global --list
user.name=Maria
user.mail=mbalcazar1470@utm.edu.ec
COMPUTEACH@DESKTOP-7MB6SH1 UCRT64 ~
$


Panel
Escribe / para buscar
Hogar
Alimentar
Principales repositorios
mbalcazar1470-ctrl/DSI_2026
¡Bienvenido a GitHub, mbalcazar1470-ctrl!
Haz cualquier pregunta o escribe @ para añadir contexto.
Preguntar
Todos los repositorios
Depurar
Agente
Crear problema
Escribe código
Git
Solicitudes de extracción
Empezando
1/3 completado
Crea tu primer proyecto de código
Iniciar un chat de Copilot
Hazle preguntas a Copilot sobre programación o GitHub y obtén ayuda instantánea con tu primer proyecto.


mbalcazar1470-ctrl / DSI_2026
Escribe / para buscar
Código
Asuntos
Solicitudes de extracción
Comportamiento
Proyectos
Wiki
Seguridad y calidad
Perspectivas
Ajustes
DSI_2026
Público
Alfiler
Mirar 0
Tenedor 0
Estrella 0
Configurar GitHub Copilot
Utiliza la función de programación en pareja con IA de GitHub para autocompletar las sugerencias mientras programas.
Comienza a usar GitHub Copilot.
Agregar colaboradores a este repositorio
Busca personas utilizando su nombre de usuario de GitHub o su dirección de correo electrónico.
Invitar a colaboradores
Configuración rápida : si ya has hecho algo así antes.
Configurado en GitHub Desktop
o
HTTPS
SSH
https://github.com/mbalcazar1470-ctrl/DSI_2026.git
Para empezar, crea un archivo nuevo o sube uno existente. Recomendamos que cada repositorio incluya un archivo README, un archivo LICENSE y un archivo .gitignore.


github.com/mbalcazar1470-ctrl/DSI_2026
Set up in GitHub Desktop
or
HTTPS
SSH
https://github.com/mbalcazar1470-ctrl/DSI_2026.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# DSI_2026" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/mbalcazar1470-ctrl/DSI_2026.git
git push -u origin main
...or push an existing repository from the command line
git remote add origin https://github.com/mbalcazar1470-ctrl/DSI_2026.git
git branch -M main
git push -u origin main
create mode 100644 Actividad_1_Maria Balcazar.pdf
COMPUTEACH@DESKTOP-7MB6SH1 UCRT64 ~/Desktop/Tareas DSI/Tarea 1 (master)
$ git remote add origin https://github.com/mbalcazar1470-ctrl/DSI_2026.git
COMPUTEACH@DESKTOP-7MB6SH1 UCRT64 ~/Desktop/Tareas DSI/Tarea 1 (master)
$ git branch -M main
COMPUTEACH@DESKTOP-7MB6SH1 UCRT64 ~/Desktop/Tareas DSI/Tarea 1 (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 691.50 KiB | 21.61 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/mbalcazar1470-ctrl/DSI_2026.git
 * [new branch]   main -> main
branch 'main' set up to track 'origin/main'.
COMPUTEACH@DESKTOP-7MB6SH1 UCRT64 ~/Desktop/Tareas DSI/Tarea 1 (main)


mbalcazar1470-ctrl / DSI_2026
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
DSI_2026
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
mbalcazar1470-ctrl commit inicial: primera version de documento
f302aa5 · 16 minutes ago
1 Commit
Actividad_1_Maria Balcazar.pdf
commit inicial: primera version de documento
16 minutes ago
README
Releases
No releases published
Create a new release
Packages
Add a README
Help people interested in this repository understand your project.
Add a README

Link de GitHub

https://github.com/mbalcazar1470-ctrl/DSI_2026

Bibliografía

- Atlassian. (s. f.). *Flujo de trabajo de Gitflow*. https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow
- Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.
- MDN Web Docs. (s. f.). *MVC*. https://developer.mozilla.org/es/docs/Glossary/MVC
- Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1
- Refactoring.Guru. (s. f.). *Patrón Factory Method*. https://refactoring.guru/es/design-patterns/factory-method
- Refactoring.Guru. (s. f.). *Patrón Observer*. https://refactoring.guru/es/design-patterns/observer
- Refactoring.Guru. (s. f.). *Patrón Singleton*. https://refactoring.guru/es/design-patterns/singleton